Pandas

This document contains useful information on how to use the Python Pandas module.

Note: The terminology included in this document and knowledge of pandas is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction

The Pandas module offers data structures and operations for manipulating numerical tables and time series. Pandas is used frequently in research and in industry for data manipulation, data science, and data visualization. Pandas builds off of Numpy, and inherits many properties like vectorized operations. Since Pandas extends the Numpy library, it is much faster than traditional Python because of the C and C++ code under the hood. In order to effectively introduce you to Pandas, we need to establish common ground regarding the terminology and software we will be using in this course.

Terminology

Pandas - Open source Python module that offers data structures and operations for manipulating numerical tables and time series for high-performance and easy to use data analysis tasks.
Series - A Pandas data structure containing a one-dimensional ndarray with axis labels.
DataFrame - The primary pandas data structure consisting of two-dimensional, size-mutable, potentially heterogeneous tabular data. It can be thought of as a dictionary container for series objects.
Inplace **-** An optional argument supplied to DataFrame and Series methods to determine if the object is mutated. The default is False, indicating that an object is returned and the original is not changed.

Getting Started

By convention, the Pandas module is imported with the following line of code; `import pandas as pd`. Pandas should have been installed with Anaconda or Miniconda, however if the import throws an error, execute `conda install pandas` from the command line .

How to follow along

Thanks to your fantastic TAs, we have created an interactive Python notebook so you do not have to copy and paste all the examples below. This notebook, which can be uploaded from Canvas, is not a normal .py file, instead it's a .ipynb file. Instructions on how to open this file can be found in the Jupyter Notebook section on page 33. To download the file click here, to access the notebook using Google Colab (It's like Google Docs for Jupyter Notebook) click here. For instructions on how to use Google Colab visit this link.

Fundamental Objects

To effectively use the Pandas module, one must understand its core. The DataFrame object is the primary Pandas data structure. A DataFrame is essentially a collection of Series, and a Series inherits many attributes and methods from the powerful NumPy array. Since the Series is the primary building block for Pandas, let's start there.

Series

In this section we will discuss creation, viewing, selection, updating, sorting, and removing values for a Series. Along the way we will mention several attributes and methods that are useful when applying Pandas to solve problems.

Series Creation

Series are primarily created using either an iterable or dictionary. Let's first look at some examples of creating a Series from an iterable.

```
>>> aList = [1,2,3,4]
>>> aSeries = pd.Series(aList)
>>> aSeries
0    1
1    2
2    3
3    4
dtype: int64
>>> aSeries.index
RangeIndex(start=0, stop=4, step=1)
>>> aSeries.values
array([1, 2, 3, 4], dtype=int64)
```

Oftentimes, we want non-numeric indices for a Series. To accomplish this, we can provide an iterable to the index parameter.

```
>>> dates = ["2020-04-08", "2020-04-09", "2020-04-10", "2020-04-11"]
>>> aSeries = pd.Series(aList, index = dates)
2020-04-08    1
2020-04-09    2
2020-04-10    3
2020-04-11    4
dtype: int64
```

Now let's look at an example of creating a Series with a dictionary.

```
>>> aDict = dict(zip(dates, aList))
>>> aDict
{'2020-04-08': 1, '2020-04-09': 2, '2020-04-10': 3, '2020-04-11': 4}
>>> aSeries = pd.Series(aDict)
>>> aSeries
2020-04-08    1
2020-04-09    2
2020-04-10    3
2020-04-11    4
dtype: int64
```

From the example above we note that the keys of the dictionary are assigned to the index attribute and the values of the dictionary are assigned to the values attribute of the Series.

Series Viewing

To view the top and bottom sections of a Series, the `.head()` and `.tail()` method are used accordingly. Both methods default to displaying 5 rows, however different values can be supplied as needed.

```
>>> aSeries = pd.Series({i : i ** 2 for i in range(100)})
>>> aSeries.head()
0     0
1     1
2     4
3     9
4    16
dtype: int64

>>> aSeries.tail()
95    9025
96    9216
97    9409
98    9604
99    9801
dtype: int64

>>> aSeries.head(10)
```

```
0     0
1     1
2     4
3     9
4    16
5    25
6    36
7    49
8    64
9    81
dtype: int64
```

Similar to a NumPy array the `.size` and `.dtype` attributes can be used to view the characteristics of the Series object.

```
>>> aSeries.size
100
>>> aSeries.dtype
dtype('int64')
```

Selection from within a Series

There are three main ways to select data from a Series, namely by label, position, and boolean indexing (masking). To select values by label we recommend using `.loc`, and to select values by position we recommend using `.iloc`.

```
>>> aSeries = pd.Series(data = [1,2,3,4], index = dates)
>>> aSeries.loc["2020-04-08"]
1
>>> aSeries.iloc[-1]
4
```

As you know, in programming there are many ways to accomplish the same thing. For a Series, it is sometimes common to select values without using `.iloc` or `.loc`. It is important to note that this only works for a Series and not a DataFrame.

```
>>> aSeries[0]
1
>>> aSeries["2020-04-11"]
4
```

Subsets of the Series can be accessed using slicing and boolean indexing just like NumPy arrays.

```
>>> aSeries.iloc[:2]
2020-04-08    1
```

```
2020-04-09    2
dtype: int64
>>> aSeries.loc["2020-04-08":"2020-04-10"]
2020-04-08    1
2020-04-09    2
2020-04-10    3
dtype: int64
```

Notice that the stop parameter for `.iloc` is exclusive while for `.loc` it's inclusive.
To retrieve a subset of a Series, we can use masking just like NumPy arrays. If we wanted aSeries with values less than 3 then we can use the following.

```
>>> aSeries[aSeries < 3]
2020-04-08    1
2020-04-09    2
dtype: int64
```

Changing a Series

A Series is a mutable data structure therefore values can be changed. To override an existing value we can use `.iloc` or `.loc`.

```
>>> aSeries.loc["2020-04-08"] = -1
>>> aSeries.iloc[1] = -2
>>> aSeries
2020-04-08   -1
2020-04-09   -2
2020-04-10    3
2020-04-11    4
dtype: int64
```

To append new values we use `.loc`.

```
>>> aSeries.loc["2020-04-12"] = 5
>>> aSeries
2020-04-08   -1
2020-04-09   -2
2020-04-10    3
2020-04-11    4
2020-04-12    5
dtype: int64
```

If we wanted to multiply every value by 2 and mutate the Series object then we can use vectorized operations and reassignment.

```
>>> aSeries = aSeries * 2
>>> aSeries
2020-04-08    -2
2020-04-09    -4
2020-04-10     6
2020-04-11     8
2020-04-12    10
dtype: int64
```

If we wanted to replace the values that meet a condition then we can use an `np.where` statement and reassignment using `.iloc`.

```
>>> np.where(aSeries < 0, 0, aSeries)
array([0, 0, 6, 8, 10], dtype=int64)
>>> aSeries.iloc[0::] = np.where(aSeries < 0, 0, aSeries)
>>> aSeries
2020-04-08    0
2020-04-09    0
2020-04-10    6
2020-04-11    8
2020-04-12   10
dtype: int64
```

Sorting a Series

Since a Series is one dimensional, sorting is inherently straight forward. To sort a Series we can use the `.sort_values` method. The main parameter of interest is `ascending` which defaults to `True`.

```
>>> aSeries.sort_values(ascending = False)
2020-04-12   10
2020-04-11    8
2020-04-10    6
2020-04-09    0
2020-04-08    0
dtype: int64
```

Deleting values from a Series

To remove a value from a Series we can call the `.drop` method. Notice that the original Series was not mutated, use `.inplace = True` to change the original Series object.

```
>>> aSeries.drop("2020-04-08")
2020-04-09    0
2020-04-10    6
2020-04-11    8
2020-04-12   10
dtype: int64
```

DataFrame

In this section we will discuss creation, viewing, selection, setting, sorting, and removing values for a DataFrame. Along the way we will mention several attributes and methods that are useful when applying Pandas to solve problems.

DataFrame Creation

DataFrames are usually read from databases or files like CSV, Excel, and JSON. Before we introduce those methods, let's explore DataFrame creation by calling the pd.DataFrame constructor. The primary parameters used to create a DataFrame using the pd.DataFrame constructor are either an iterable or a dictionary. Let's first look at some examples of creating a DataFrame from an iterable.

```
>>> grades = np.array([["Exam 1", "Exam 2", "Exam 3"], \
                       [      90,       87,       96], \
                       [      79,      100,       92], \
                       [      98,       60,       74]])

>>> df = pd.DataFrame(data = grades)
>>> df
        0       1       2
0  Exam 1  Exam 2  Exam 3
1      90      87      96
2      79     100      92
3      98      60      74
```

Notice that by default, Pandas assigns the position values as the labels for the index and columns. In this case, we want Exam 1, Exam 2, and Exam 3 to be the column labels instead of the first row. We can do this by leveraging the columns parameter.

```
>>> df = pd.DataFrame(data = grades[1:], columns = grades[0])
>>> df
  Exam 1 Exam 2 Exam 3
0     90     87     96
1     79    100     92
```

```
2     98     60     74
```

If we wanted to change the values for the index, we can do so by either reassigning the index attribute or providing the index parameter.

```
>>> names = ["George B.", "Calvin J.", "Chris P."]
>>> df.index = names
>>> df
           Exam 1 Exam 2 Exam 3
George B.      90     87     96
Calvin J.      79    100     92
Chris P.       98     60     74
>>> pd.DataFrame(data = grades[1:], columns = grades[0], \
                 index = names)
           Exam 1 Exam 2 Exam 3
George B.      90     87     96
Calvin J.      79    100     92
Chris P.       98     60     74
```

Now let's look at some examples of creating a DataFrame from iterables and dictionaries.

```
>>> names = ["George B.", "Calvin J.", "Chris P."]
>>> grades = {'Exam 1': [90, 79, 98],
              'Exam 2': [87, 100, 60],
              'Exam 3': [96, 92, 74]}
>>> pd.DataFrame(data = grades, index = names)
           Exam 1  Exam 2  Exam 3
George B.      90      87      96
Calvin J.      79     100      92
Chris P.       98      60      74
>>> grades = {'Exam 1': \
                  {'George B.': 90, 'Calvin J.': 79, 'Chris P.': 98},\
              'Exam 2': \
                 {'George B.': 87, 'Calvin J.': 100, 'Chris P.': 60},\
              'Exam 3': \
                  {'George B.': 96, 'Calvin J.': 92, 'Chris P.': 74},\
             }
>>> df = pd.DataFrame(data = grades)
>>> df
           Exam 1  Exam 2  Exam 3
George B.      90      87      96
Calvin J.      79     100      92
```

```
Chris P.       98      60      74
```

DataFrame Viewing

Identical to Series, to view the top and bottom sections of a DataFrame, the `.head()` and `.tail()` method are used accordingly. Both methods default to displaying 5 rows, however different values can be supplied as needed.

```
>>> df = pd.DataFrame([[i**2, i**3] for i in range(100)], columns =
[“x**2”, “x**3”])
>>> df.head()
   x**2  x**3
0     0     0
1     1     1
2     4     8
3     9    27
4    16    64

>>> df.tail(10)

    x**2    x**3
90  8100  729000
91  8281  753571
92  8464  778688
93  8649  804357
94  8836  830584
95  9025  857375
96  9216  884736
97  9409  912673
98  9604  941192
99  9801  970299
```

Similar to a NumPy array the `.shape`, `.size`, and `.dtypes` attributes can be used to view the characteristics of the DataFrame object.

```
>>> df.shape
(100, 2)
>>> df.size
200
```

```
>>> df.dtypes
x**2    int64
x**3    int64
dtype: object
```

Finally, to view summary statistics of a DataFrame we can call the `.describe()` method and round the results to two decimal places.

```
>>> df.describe().round(2)
          x**2       x**3
count   100.00     100.00
mean   3283.50  245025.00
std    2968.17  280457.58
min       0.00       0.00
25%     612.75   15174.75
50%    2450.50  121324.50
75%    5513.25  409386.75
max    9801.00  970299.00
```

Selection from within a DataFrame

When selecting data from a DataFrame, we can select subsets of the columns and rows. To select a subset of a DataFrame's columns we use the following syntax.

```
>>> df = pd.DataFrame(data = [["CS2316", 150, 3.15],\
                              ["CS1331", 900, 2.91],\
                              ["ISyE2027",200,2.86],\
                              ["ISyE2028",200, 3.11],\
                              ["MATH2603", 150, 3.17]],\
                      columns = ["Course", "Enrollment", "Avg_GPA"])
>>> df
      Course  Enrollment  Avg_GPA
0    CS2316         150     3.15
1    CS1331         900     2.91
2  ISyE2027         200     2.86
3  ISyE2028         200     3.11
4  MATH2603         150     3.17

>>> df["Course"]
0      CS2316
1      CS1331
2    ISyE2027
3    ISyE2028
```

```
4    MATH2603
Name: Course, dtype: object

>>> type(df["Course"])
pandas.core.series.Series

>>> df[["Course", "Avg_GPA"]]
     Course   Avg_GPA
0    CS2316      3.15
1    CS1331      2.91
2  ISyE2027      2.86
3  ISyE2028      3.11
4  MATH2603      3.17
```

Notice that when one column is selected from a DataFrame, the return type is a Series. Now when selecting rows from a DataFrame, we can use both `.loc` and `.iloc`. Currently both methods would return the same results so let's make the Course column the index. *note the use of the `inplace` parameter, defined above under terminology*

```
>>> df.set_index("Course", inplace = True)
>>> df
          Enrollment  Avg_GPA
Course
CS2316           150     3.15
CS1331           900     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11
MATH2603         150     3.17
```

Now if we want to retrieve the average GPA for ISyE 2027, we can first select the `"ISyE2027"` row which returns a Series then select `"Avg_GPA"` column. Or we can first select the `"Avg_GPA"` column then select the `"ISyE2027"` row.

```
>>> df.loc["ISyE2027"]
Enrollment    200.00
Avg_GPA         2.86
Name: ISyE2027, dtype: float64
>>> df.loc["ISyE2027"].loc["Avg_GPA"]
2.86
>>> df["Avg_GPA"].loc["ISyE2027"]
2.86
```

Finally, we can combine both methods into simple concise syntax that is similar to indexing and slicing ndarrays.

```
>>> df.loc["ISyE2027", "Avg_GPA"]
2.86
>>> df.iloc[2, -1]
2.86
>>> df.iloc[2:4, :]
          Enrollment  Avg_GPA
Course
ISyE2027         200     2.86
ISyE2028         200     3.11

>>> df.loc["ISyE2027":"ISyE2028", "Enrollment"]
Course
ISyE2027    200
ISyE2028    200
Name: Enrollment, dtype: int64
```

The last selection method is boolean indexing (masking). Since DataFrames are a collection of Series, we can exploit the vectorized nature of series to perform element-wise comparisons. For example, if we wanted to create a DataFrame containing all rows and columns that have enrollment greater than 150 and average GPA less than 3 we can use the following.

```
>>> df[df["Enrollment"] > 150]
          Enrollment  Avg_GPA
Course
CS1331           900     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11

>>> df[df["Avg_GPA"] < 3]
          Enrollment  Avg_GPA
Course
CS1331           900     2.91
ISyE2027         200     2.86

>>> df[(df["Enrollment"] > 150) & (df["Avg_GPA"] < 3)]
          Enrollment  Avg_GPA
Course
CS1331           900     2.91
ISyE2027         200     2.86
```

An additional technique used to filter a DataFrame is to apply an `isin` comparison for each item then use the booleans returned to filter the results. For example, to return only ISyE courses we can check to see if each value of the index is in `["ISyE2027", "ISyE2028"]`.

```
>>> df[df.index.isin(["ISyE2027", "ISyE2028"])]
          Enrollment  Avg_GPA
Course
ISyE2027         200     2.86
ISyE2028         200     3.11
```

Changing data in a DataFrame

Adding or Replacing Row Values

A DataFame is a mutable data structure therefore values can be changed. To add or override an existing value we can use `.iloc` or `.loc` just like for Series. The difference is that a DataFrame by definition has more than one column thus if the values for a row change then an iterable or dictionary should be supplied. If the index doesn't exist, we are adding a row. If the index exists we are mutating a row. Let's first start with changing a single value, in this example the enrollment of `"CS1331"` to `850`.

```
>>> df.loc["CS1331", "Enrollment"] = 850
>>> df
          Enrollment  Avg_GPA
Course
CS2316           150     3.15
CS1331           850     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11
MATH2603         150     3.17
```

Now let's change all the values for a given row, say `"MATH2603"`.

```
>>> df.iloc[-1, :] = [100, 3.15]
>>> df
          Enrollment  Avg_GPA
Course
CS2316           150     3.15
CS1331           850     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11
MATH2603         100     3.15
```

Add a row by using the same syntax, but only the index is needed inside the square brackets.

```
>>> df.loc[“ISyE3232”] = [150, 3.14]
>>> df
          Enrollment  Avg_GPA
Course
CS2316           150     3.15
CS1331           850     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11
MATH2603         100     3.15
ISyE3232         150     3.14
```

Adding or Replacing Columns

Frequently we want to generate new features that are derived from existing ones. Let's say we define courses as large if the enrollment is greater than or equal to 150. We can create a `"Large"` column using the following:

```
>>> df["Large"] = df["Enrollment"] >= 200
>>> df
          Enrollment  Avg_GPA  Large
Course
CS2316           150     3.15  False
CS1331           850     2.91   True
ISyE2027         200     2.86   True
ISyE2028         200     3.11   True
MATH2603         100     3.15  False
ISyE3232         150     3.14  False
```

As discussed previously in this course, binary values like booleans are commonly represented with `0` and `1`. To convert the values of the `"Large"` column to integers we can use the following:

```
>>> df["Large"] = df["Large"].astype(int)
>>> df
          Enrollment  Avg_GPA  Large
Course
CS2316           150     3.15      0
CS1331           850     2.91      1
ISyE2027         200     2.86      1
ISyE2028         200     3.11      1
MATH2603         100     3.15      0
ISyE3232         150     3.14      0
```

To overwrite an existing column, we simply use the same syntax on an existing column.

Sorting a DataFrame

Similar to Series, DataFrames can also be sorted using the `.sort_values` method. In addition to the `ascending` parameter another important parameter is `by` which denotes which column is used for sorting.

```
>>> df.sort_values(by = "Avg_GPA", ascending = False, inplace = False)
          Enrollment  Avg_GPA  Large
Course
CS2316           150     3.15      0
MATH2603         100     3.15      0
ISyE3232         150     3.14      0
ISyE2028         200     3.11      1
CS1331           850     2.91      1
ISyE2027         200     2.86      1
```

Deleting values from a DataFrame

In the process of cleaning data, it is very common to remove columns and rows from the DataFrame. To accomplish this, we can call the `.drop` method and provide value(s) to be removed and the axis on which the values are held. In the example above if we wanted to remove the `"Large"` column we can use the following:

```
>>> df.drop(["Large"], axis = 1)
          Enrollment  Avg_GPA
Course
CS2316           150     3.15


CS1331           900     2.91
ISyE2027         200     2.86
ISyE2028         200     3.11
MATH2603         150     3.17
ISyE3232         150     3.14
```

The same concept applies to remove rows. Suppose we want to remove rows with index label `"CS2316"` and `"CS1331"`

```
>>> df.drop(["CS2316", "CS1331"], axis = 0)
          Enrollment  Avg_GPA  Large
Course
ISyE2027         200     2.86      1
ISyE2028         200     3.11      1
MATH2603         150     3.17      0
```

```
ISyE3232         150     3.14      0
```

Removing Duplicate Rows

A common occurrence when cleaning data is the removal of duplicate rows. While using the `.drop` method is a viable way to target specific rows to be removed, we can use the `.drop_duplicates` method to remove any duplicate rows. Optional parameters can be added: `subset` to only consider duplicates in certain columns expressed in a list, `keep` to choose whether to keep the first or last occurrence of duplicates, or `inplace` to modify the DataFrame inplace. Suppose we want to remove duplicate rows in the Course column:

```
>>> df
      Course  Enrollment  Avg_GPA
0    CS2316         150     3.15
1    CS1331         900     2.91
2  ISyE2027         200     2.86
3  ISyE2027         200     3.11
4  MATH2603         150     3.17

>>> df.drop_duplicates(subset = [“Course”])
      Course  Enrollment  Avg_GPA
0    CS2316         150     3.15
1    CS1331         900     2.91
2  ISyE2027         200     2.86
3  MATH2603         150     3.17
```

Counting Unique Rows

The complement to removing duplicate rows is identifying unique rows. The `.nunique()` method counts the number of distinct elements in a specified axis, returning a Series with the number of unique elements. Optional parameters can be added to ignore NaN values. Suppose we want to count unique rows:

```
>>> df.nunique(axis = 0)
Course        4
Enrollment    3
Avg_GPA       5
```

DataFrame File I/O

Reading

As mentioned previously, DataFrames are typically constructed from sources like tables in a database or data exchange formats like JSON, CSV, and Excel. We will primarily focus on CSV and Excel files, but if you're interested we've provided links for SQL and JSON in the additional methods section.

pd.read_csv

The pd.read_csv method allows users to construct a DataFrame from a csv file by providing the path (can be URL), delimiter, index column, and other parameters. Suppose we have a `"courses.csv"` present in our current working directory.

```
>>> courses = pd.read_csv("<path>/courses.csv", delimiter = ",",
index_col = 0)
>>> courses
           Enrollment  Avg_GPA  Large
Course
CS2316         150.0     3.15    0.0
CS1331         850.0     2.91    1.0
ISyE2027       200.0     2.86    1.0
ISyE2028       200.0     3.11    1.0
MATH2603       100.0     3.15    0.0
CS2803          50.0      NaN    0.0
ISyE3803        50.0      NaN    0.0
```

As with other methods of reading CSV's, `delimiter` defaults to a comma.

pd.read_excel

A notable strength of pandas is its ability to read Excel files as well. Recall that the csv module struggled with any file that has the .xlsx extension. With pandas, we can use pd.read_excel() to easily read excel files, even ones containing multiple sheets. Given the excel file linked here, we can open the file and read each sheet into a dataframe.

```
>>> countries = pd.read_excel("<path>/olympics.xlsx", sheet_name = "dictionary")
>>> countries.head()
          Country Code  Population           GDP
0     Afghanistan  AFG  32526562.0  1.933129e+10
1         Albania  ALB   2889167.0  1.139839e+10
2         Algeria  ALG  39666519.0  1.668386e+11
3  American Samoa*  ASA     55538.0           NaN
4         Andorra  AND     70473.0           NaN
```

```
>>> summer = pd.read_excel("<path>/olympics.xlsx", sheet_name = "summer", header = 1)
>>> summer.head()
   0       1         2          3                  4        5       6                          7       8
0  Year    City     Sport  Discipline            Athlete  Country  Gender                     Event   Medal
1  1896  Athens  Aquatics    Swimming       HAJOS, Alfred      HUN     Men             100M Freestyle    Gold
2  1896  Athens  Aquatics    Swimming   HERSCHMANN, Otto      AUT     Men             100M Freestyle  Silver
3  1896  Athens  Aquatics    Swimming    DRIVAS, Dimitrios      GRE     Men  100M Freestyle For Sailors  Bronze
4  1896  Athens  Aquatics    Swimming  MALOKINIS, Ioannis      GRE     Men  100M Freestyle For Sailors    Gold
```

pd.ExcelFile

As is typical in programming, there is another way to do the exact same thing as above. We can use pd.ExcelFile.parse() to achieve the same results after making an ExcelFile object using pd.ExcelFile().

```
>>> fin = pd.ExcelFile("<path>\olympics.xlsx")
>>> countries = fin.parse(sheet_name = "dictionary")
>>> countries.head()
           Country Code  Population           GDP
0      Afghanistan  AFG  32526562.0  1.933129e+10
1          Albania  ALB   2889167.0  1.139839e+10
2          Algeria  ALG  39666519.0  1.668386e+11
3  American Samoa*  ASA     55538.0           NaN
4          Andorra  AND     70473.0           NaN
```

Writing

df.to_csv

We can also use pandas to write information into various file formats. First, let's see how to use df.to_csv() to write a pandas dataframe into a csv file. Note the `index = True` parameter, which will write the csv using the dataframe's indices.

```
>>> courses.to_csv("<path>/output.csv", index = True)
*this creates a csv identical to our original:

Course,Enrollment,Avg_GPA,Large
```

```
CS2316,150.0,3.15,0.0
CS1331,850.0,2.91,1.0
ISyE2027,200.0,2.86,1.0
ISyE2028,200.0,3.11,1.0
MATH2603,100.0,3.15,0.0
CS2803,50.0,,0.0
ISyE3803,50.0,,0.0
```

pd.ExcelWriter

We can also use pandas to write to Excel files. First, we create an ExcelWriter object using pd.ExcelWriter(), similarly to creating a csv writer object from the CSV module.

```
>>> writer = pd.ExcelWriter("<path>/output.xlsx")
```

df.to_excel

After creating our writer object, we can use df.to_excel() to write our dataframe into an excel file.

```
>>> summer.to_excel(writer)
```

We can also write multiple sheets to an excel file at once.

```
>>> countries.to_excel(writer, sheet_name = "dictionary")
>>> summer.to_excel(writer, sheet_name = "summer")
```

This will write an excel file identical to our original olympics.xlsx

Finally to save the changes to disk, you must either use a context manager or call the `.save` method on the writer object. The following code consolidates everything together:

```
>>> writer = pd.ExcelWriter("<path>/output.xlsx")
>>> countries.to_excel(writer, sheet_name = "dictionary")
>>> summer.to_excel(writer, sheet_name = "summer")
>>> writer.save()
```

Additional Methods

JSON	Reading	Documentation Example
	Writing	Documentation Example
SQL	Reading	Documentation Example
	Writing	Documentation
HTML	Reading	Documentation Example
	Writing	Documentation Example

Missing Data

As discussed in the NumPy handout, NaN values are commonly found when reading data. The examples below show you to remove, fill, and check for NaN values. To inject NaNs into our previous DataFrame, suppose two new courses were added, CS2803 and ISyE3803, both of which have an unknown average GPA.

```
>>> df.loc["CS2803"] = [50, np.nan, 0]
>>> df.loc["ISyE3803"] = [50, np.nan, 0]
>>> df
          Enrollment  Avg_GPA  Large
Course
CS2316         150.0     3.15    0.0
CS1331         850.0     2.91    1.0
ISyE2027       200.0     2.86    1.0
ISyE2028       200.0     3.11    1.0
MATH2603       100.0     3.15    0.0
CS2803          50.0      NaN    0.0
ISyE3803        50.0      NaN    0.0
```

Now if we want to remove all rows that contain a NaN value we can use the following:

```
>>> df.dropna()
          Enrollment  Avg_GPA  Large
```

```
Course
CS2316         150.0     3.15    0.0
CS1331         850.0     2.91    1.0
ISyE2027       200.0     2.86    1.0
ISyE2028       200.0     3.11    1.0
MATH2603       100.0     3.15    0.0
```

If we want to fill NaN values with a desired value we can use the .fillna method.

```
>>> df.fillna(0)
          Enrollment  Avg_GPA  Large
Course
CS2316         150.0     3.15    0.0
CS1331         850.0     2.91    1.0
ISyE2027       200.0     2.86    1.0
ISyE2028       200.0     3.11    1.0
MATH2603       100.0     3.15    0.0
CS2803          50.0     0.00    0.0
ISyE3803        50.0     0.00    0.0
```

Finally, to check if a value is NaN we can use the pd.isna function.

```
>>> pd.isna(df)
          Enrollment  Avg_GPA  Large
Course
CS2316         False    False  False
CS1331         False    False  False
ISyE2027       False    False  False
ISyE2028       False    False  False
MATH2603       False    False  False
CS2803         False     True  False
ISyE3803       False     True  False
```

Operations

Now that we know how to create, add, and manipulate values for a DataFrame and Series let's move into operations that are used to summarize, modify, and explore the data. For the examples ahead we will use the following DataFrames.

```
>>> grades = {
              'Exam 1': \
                  {'George B.': 90, 'Calvin J.': 79, 'Chris P.': 98},
              'Exam 2': \
```

```
                    {'George B.': 87, 'Calvin J.': 100, 'Chris P.': 60},
                'Exam 3': \
                    {'George B.': 96, 'Calvin J.': 92, 'Chris P.': 74},
                }
>>> grades = pd.DataFrame(data = grades)
>>> grades
            Exam 1  Exam 2  Exam 3
George B.       90      87      96
Calvin J.       79     100      92
Chris P.        98      60      74
```

Aggregates

See this link to Pandas documentation displaying all statistical functions that can be applied to DataFrames. Below we will show a quick example applying the `mean` method across both axes. The same concept applies for other aggregate functions such as `max`, `min`, `sum`, `count`, and many others.

Using the `grades` DataFrame defined above, if we want to add a column titled `"Final Grade"`, we could do so by applying the mean function across each row.

```
>>> grades["Final Grade"] = grades.mean(axis = 1).round(2)
>>> grades
           Exam 1  Exam 2  Exam 3  Final Grade
George B.       90      87      96        91.00
Calvin J.       79     100      92        90.33
Chris P.        98      60      74        77.33
```

Now if we want to compute the average for all the columns then we can apply the mean function across each column.

```
>>> grades.loc["Average Grade"] = grades.mean(axis=0).round(2)
>>> grades
                Exam 1  Exam 2  Exam 3  Final Grade
George B.         90.0   87.00   96.00        91.00
Calvin J.         79.0  100.00   92.00        90.33
Chris P.          98.0   60.00   74.00        77.33Average Grade
89.0   82.33   87.33        86.22
```

Apply

Sometimes, we want to apply a function that is custom to the data set at hand. An example of this can be shown using the `grades` DataFrame. Suppose we want to create a `"Letter Grade"` column denoting the final course grade. First let's define a function.

```
>>> def letter_grade(final_grade):
...       if final_grade >= 90:
...           return "A"
...       elif final_grade >= 80:
...           return "B"
...       elif final_grade >= 70:
...           return "C"
...       elif final_grade >= 60:
...           return "D"
...       return "F"
```

Now with the function defined we can call the `.apply` method on `axis = 1`

```
>>> grades["Letter Grade"] = grades.apply(lambda x :
letter_grade(x["Final Grade"]), axis = 1)
>>> grades
           Exam 1  Exam 2  Exam 3  Final Grade Letter Grade
George B.    90.0   87.00   96.00        91.00            A
Calvin J.    79.0  100.00   92.00        90.33            A
Chris P.     98.0   60.00   74.00        77.33            C
Averages     89.0   82.33   87.33        86.22            B
```

In this case, each `x` represents a row in the DataFrame because we are applying the function horizontally, therefore we can access the `"Final Grade"` value for each row and calculate the letter grade by calling the `letter_grade` function.

If we want to sort a column that has a list, we can use .apply(sorted) to accomplish that.

String Methods

Suppose we reset the index to the grades DataFrame.

```
>>> grades = grades.reset_index()
>>> grades.columns
Index(['index', 'Exam 1', 'Exam 2', 'Exam 3', 'Final Exam', 'Letter
Grade'], dtype='object')

>>> grades.rename(columns = {"index" : "Name"}, inplace = True)
>>> grades

        Name  Exam 1  Exam 2  Exam 3  Final Exam Letter Grade
0  George B.    90.0   87.00   96.00       91.00            A
```

```
1  Calvin J.    79.0  100.00   92.00       90.33           A
2   Chris P.    98.0   60.00   74.00       77.33           C
3   Averages    89.0   82.33   87.33       86.22           B
```

Now suppose we want to know what rows contain a letter grade of 'A'. To accomplish this we can use the .str functionality available for Series. A list of all string methods can be found here. For this example, we will be seeing which grade value `contains` the letter A, with all NaN values being replaced with False.

```
>>> grades["Grade"].str.contains("A", na = False)
0    True
1    True
2    False
3    True
Name: Grade, dtype: boollast
```

Now further suppose we want to construct two columns for the first name and last initial for each student. or this example we will demonstrate how to `split` the values of a column by a given delimiter, in this case a space.

```
>>> grades["Name"].str.split(" ")
0    [George, B.]
1    [Calvin, J.]
2     [Chris, P.]
3      [Averages]
Name: Name, dtype: object
```

Notice that each value is now a list within the Series values which isn't exactly what we wanted. Thankfully, the .split method has a parameter `expand` which is clearly explained with an example.

```
>>> names = grades["Name"].str.split(" ", expand = True)
>>> names
          0     1
0    George    B.
1    Calvin    J.
2     Chris    P.
3  Averages  None
```

Now to clean the `names` DataFrame a little and add it to the `grades` DataFrame we can call the `.replace` method to replace a period with an empty string. Note that the `.replace` method can use regular expressions resembling the `re.sub` function.

```
>>> names[1] = names[1].str.replace(r"\.","")
```

```
>>> names
          0     1
0    George     B
1    Calvin     J
2     Chris     P
3  Averages  None
```

Finally, we can create the “First Name” and “Last Initial” column to the original grades DataFrame.

```
>>> grades["First Name"], grades["Last Name"] = names.values.T
>>> grades
        Name  Exam 1  Exam 2  Exam 3  Final Exam Letter Grade First Name Last Initial
0  George B.    90.0   87.00   96.00       91.00            A     George            B
1  Calvin J.    79.0  100.00   92.00       90.33            A     Calvin            J
2   Chris P.    98.0   60.00   74.00       77.33            C      Chris            P
3   Averages    89.0   82.33   87.33       86.22            B   Averages         None
```

Now, if we wanted to reverse splitting up these names, we could use the .cat method to combine two string columns into one. `.cat` takes in an iterable of strings, “others”, and something to separate them, “sep”. We can use this to either create a new column or change an existing one.

```
>>>grades["Full Name"] = grades["First Name"].str.cat
(others = [grades["Last Initial"]], sep = " ")
>>>grades[["Full Name", "First Name", "Last Initial"]]

   Full Name  First Name  Last Initial
0   George B      George             B
1   Calvin J      Calvin             J
2    Chris P       Chris             P
```

Groupby

Groupby operations can be performed on a DataFrame to summarize and describe clusters with identical features within the dataset. Consider the following summer DataFrame shown in the DataFrame File I/O section.

```
>>> summer.columns
Index(['Year', 'City', 'Sport', 'Discipline', 'Athlete', 'Country', 'Gender',  'Event', 'Medal'], dtype='object')
>>> summer.head()
       Year    City      Sport          Discipline  ... Country Gender                     Event   Medal
0      1896  Athens   Aquatics            Swimming  ...     HUN    Men             100M Freestyle    Gold
1      1896  Athens   Aquatics            Swimming  ...     AUT    Men             100M Freestyle  Silver
2      1896  Athens   Aquatics            Swimming  ...     GRE    Men  100M Freestyle For Sailors  Bronze
3      1896  Athens   Aquatics            Swimming  ...     GRE    Men  100M Freestyle For Sailors    Gold
4      1896  Athens   Aquatics            Swimming  ...     GRE    Men  100M Freestyle For Sailors  Silver
```

To start with a simple groupby, let’s look at the total Olympic medals for each country.

```
>>> total_medals = summer.groupby("Country")["Medals"].count()

>>> total_medals.sort_values(ascending = False)
Country
USA    4585
URS    2049
GBR    1720
FRA    1396
GER    1305
       ...
GUY       1
GUA       1
DJI       1
GRN       1
BRN       1
Name: Medal, Length: 147, dtype: int64
```

Now to perform a groupby using more than one column we can groupby country then by gender to find the total medals for each country then each gender.

```
>>> total_medals =
summer.groupby(["Country","Gender"])["Medal"].count()
>>> total_medals.iloc[35:40]
Country  Gender
CHI      Men        32
         Women       1
CHN      Men       270
         Women     537
CIV      Men         1
Name: Medal, dtype: int64

>>> total_medals.sort_values(ascending = False)
Country  Gender
USA      Men       3208
URS      Men       1476
GBR      Men       1412
USA      Women     1377
FRA      Men       1254
                   ...
BRN      Women        1
SIN      Men          1
BOT      Men          1
BOH      Women        1
GAB      Men          1
```

```
Name: Medal, Length: 236, dtype: int64
```

Notice the syntax is `DataFrame.groupby([column1, column2, ...])[aggregate column1, aggregate column2, ...] regate()`
For example, let's find the average number of medals for each gender using the previous total_medals DataFrame.

```
>>> df = total_medals.reset_index()
>>> df
    Country Gender  Medal
0       AFG    Men      2
1       AHO    Men      1
2       ALG    Men     12
3       ALG  Women      3
4       ANZ    Men     27
..      ...    ...    ...
231     YUG  Women     62
232     ZAM    Men      2
233     ZIM  Women     23
234     ZZX    Men     45
235     ZZX  Women      3

>>> df.groupby("Gender")["Medal"].mean()
Gender
Men      161.304965
Women     88.600000
Name: Medal, dtype: float64
```

In this example, `"Gender"` is the group column, `"Medal"` is the column on which the mean is applied for each group, and `mean` is the aggregate of choice. In the example above we are only using one aggregate column, sometimes it's useful to perform more than one aggregate on more than one column. In cases like this we can use the `.aggregate` or `.agg` method (which both represent the same method) on the groupby object. For example, let's find the average number of medals for each gender as well as the total number of countries represented by each gender.

```
>>> df.groupby("Gender").aggregate({"Medal":"mean", "Country" :
"count"})
              Medal  Country
Gender
Men      161.304965      141
```

```
Women     88.600000       95
```

You can also create a DataFrame where the cell values are a list of all of the values in the group by using .agg(list)

For more information about groupby, visit this link showing more specialized and powerful uses of groupby.

Combining DataFrames

In order to streamline a process, sometimes data needs to be combined from multiple files or sources. Pandas is an excellent platform to perform this task for reasonably sized data.

Concatenate

Visit this URL and explore the given data files. After observing some of the files, it is easy to see that all files have the same columns, yet there are 30 files! If we want to consolidate each file into one DataFrame we can use pd.concat.

First let's construct a list of all DataFrames.

```
>>> dfs = []
>>> for day in range(1, 32):
...     day = str(day) if day > 9 else "0" + str(day)
...     URL =
f"https://raw.githubusercontent.com/j-on-son/Data/master/ISyE3803/R
elay%20Bikes/March%20Data%20for%20rentals%20in%20vs.%20out%20by%20s
tation/hub_stats_relay_bike_share_03_{day}_2018-03_{day}_2018.csv"
...     df = pd._csv(URL)
...     df["Day"] = day
...     dfs.append(df)
```

Now we can concatenate the list of DataFrames vertically.

```
>>> pd.concat(dfs, axis = 0)
                                                  Name  ...  Day
0              VIRTUAL HUB - ATLANTA BICYCLE COALITION  ...   01
1                           VIRTUAL HUB - CANDLER PARK  ...   01
2        VIRTUAL HUB - BROWNWOOD PARK RECREATION CENTER  ...   01
3    TWO WHEEL VALET-CARE WALK IN HER SHOES- VIRTUA...  ...   01
4                                      BROAD & MITCHELL  ...   01
..                                                  ...  ...  ...
118                                VIRGINIA-HIGHLAND  ...   31
119                               INMAN PARK VILLAGE  ...   31
120                                      GRANT PARK  ...   31
121                          HIGHLAND - FREEDOM TRAIL  ...   31
122                                  PONCE - HIGHLAND  ...   31

[3813 rows x 16 columns]
```

Writing to Other Datatypes

Dictionaries

If you want to write your DataFrame out to a dictionary, use df.to_dict(). It will default to having the column indices as the main dictionary keys, and then create a sub-dictionary with the row indices and cell values. If you want it the other way around (row indices as the main dictionary keys and a sub-dictionary with the column indices and cell values), you will need to use the orient = "index" argument. See more information here (an example from this site is shown below).

```
>>> df = pd.DataFrame({'col1': [1, 2],
...                    'col2': [0.5, 0.75]},
...                   index=['row1', 'row2'])
>>> df
      col1  col2
row1     1  0.50
row2     2  0.75
>>> df.to_dict()
{'col1': {'row1': 1, 'row2': 2}, 'col2': {'row1': 0.5, 'row2': 0.75}}
```

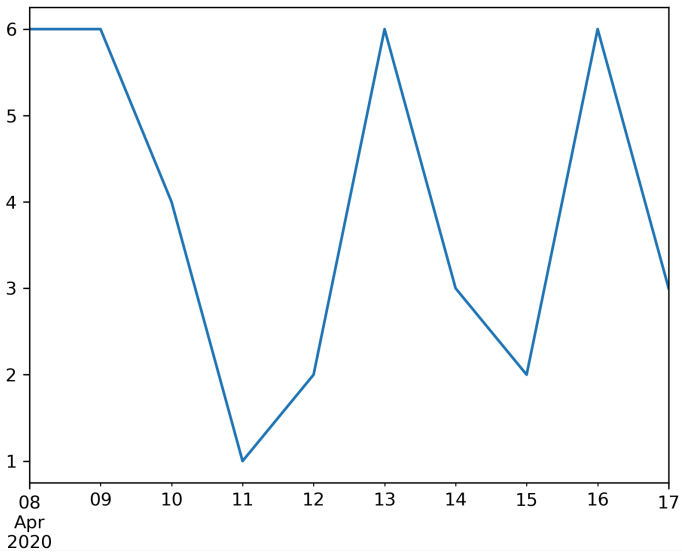
Plotting

Every Pandas Series and DataFrame has the `.plot` method which is used to visualize patterns in the data. The parameters of interest are `x`, `y`, and `kind`. Below we will highlight some of the basic plots that may be helpful for simple data visualization. But first, let's define a Series with uniformly distributed random values indexed by dates.

```
>>> aSeries = pd.Series(index = pd.date_range("20200408", periods =
10), data = np.random.randint(1,7, 10))
>>> aSeries
2020-04-08    6
2020-04-09    6
2020-04-10    4
2020-04-11    1
2020-04-12    2
2020-04-13    6
2020-04-14    3
2020-04-15    2
2020-04-16    6
2020-04-17    3
Freq: D, dtype: int32
```

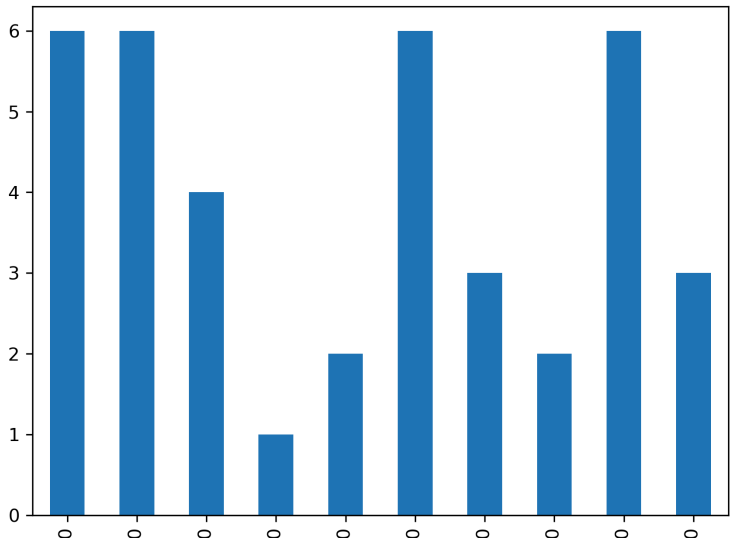
Line

```
>>> aSeries.plot(x = aSeries.index, y = aSeries.values)
```



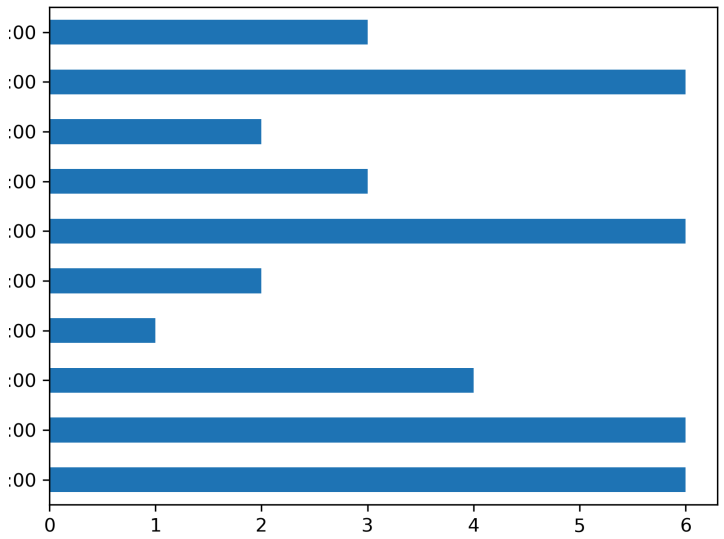
Bar

```
>>> aSeries.plot(x = aSeries.index, y = aSeries.values, kind="bar")
```



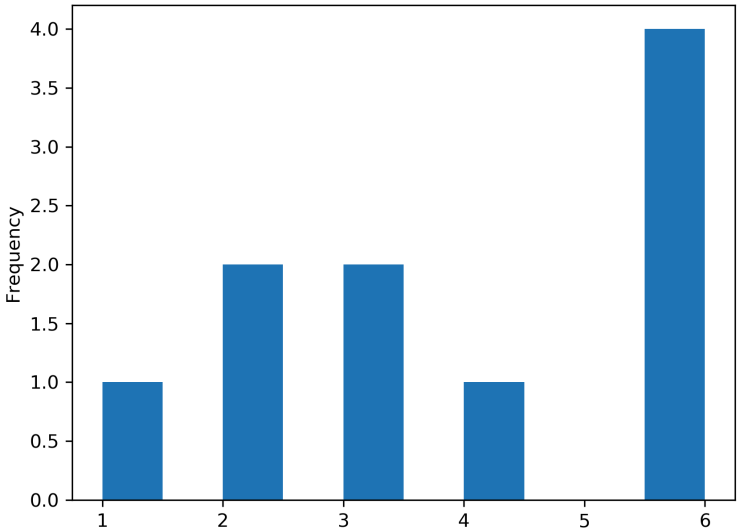
Barh

```
>>> aSeries.plot(x = aSeries.index, y = aSeries.values,
kind="barh")
```



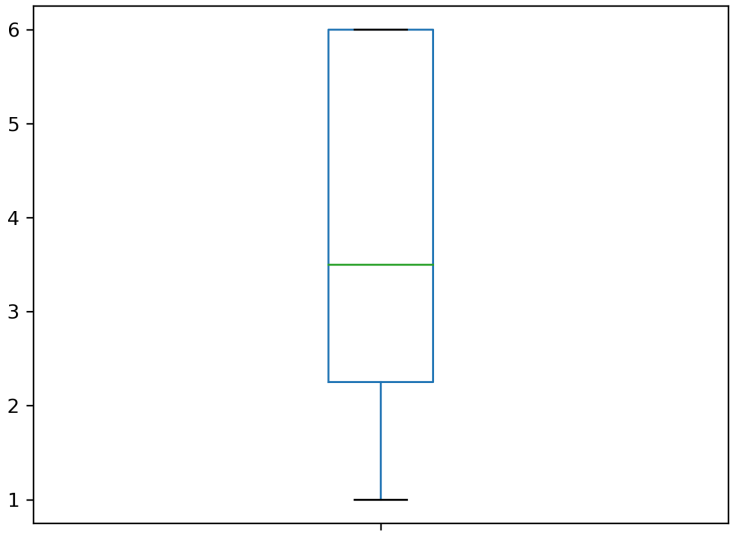
Hist

```
>>> aSeries.plot(x = aSeries.index, y = aSeries.values,
kind="hist")
```



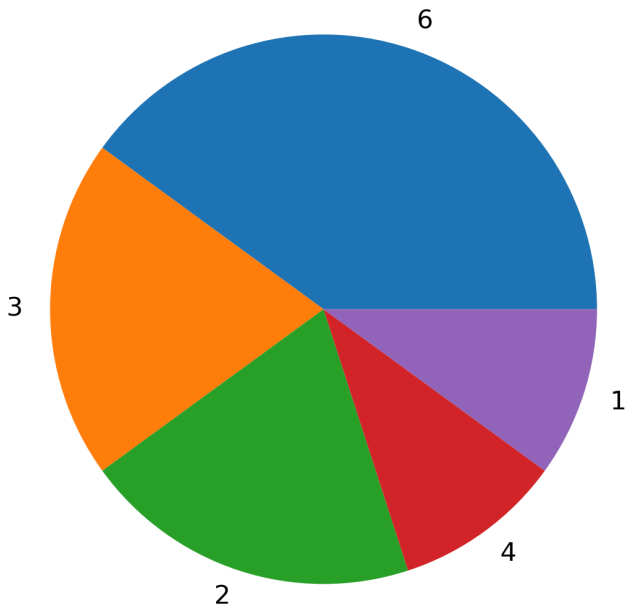
Box

```
>>> aSeries.plot(x = aSeries.index, y = aSeries.values, kind="box")
```



Pie

```
>>> percent_occurances = aSeries.value_counts()/10 * 100
>>> percent_occurances.plot(kind = "pie")
```



Visualization Libraries

A separate handout on Data Visualization covers other ways to visualize Pandas and other types of data. The libraries listed below are only a few of the many libraries available.

Matplotlib	Documentation Examples
Seaborn	Documentation Examples
Plotly	Documentation Examples
JavaScript's D3 (super powerful)	Documentation Examples

Jupyter Notebook

Jupyter Notebook is a useful tool that allows us to create and share documents that contain not just text, but live code, equations, and visualizations. It supports over 40 different programming languages (including Python), it's capable of generating graphs and other data visualization tools, and it's great for presenting blocks of code in an easy-to-read format. In many industries, it's a popular tool to use when performing data analysis with Pandas.

Installation

To install jupyter notebooks, you need to install the jupyter library. Assuming that you have installed Anaconda/Miniconda on your computer, in command prompt/terminal, type:

```
>>> jupyter notebook
```

In previous units, after we've installed a library, we've imported it using import statements at the top of our python files. However, we won't be doing that for the jupyter library, since we use it primarily as an application to display our code and its results.

To launch the Jupyter Notebook application, in your command prompt/Terminal, navigate to the desired directory, then type `jupyter notebook`, and after a few seconds, a new window should open in your browser with Jupyter Notebooks showing you a list of all of the files in the directory that you opened it in.

*__NOTE__: after you have opened Jupyter Notebook, your command line will be occupied until you close the window that was running Jupyter. You can also run Ctrl + C to terminate the Jupyter application and return to your regular command prompt/Terminal.

Jupyter Notebook Files (.ipynb)

For this class, the type of file we will be using in Jupyter Notebook will most likely have the extension .ipynb, which stands for iPython Notebook. When you open a file of this type, you might see why it's named this way, since the display is similar to the iPython interface. These files are really useful, since they can be exported to HTML, pdf, and LaTeX formats.

To open one, open the Jupyter application from the command line using the `jupyter notebook` command, and a new browser window should open showing you the Jupyter Application.

To create a new .ipynb file from inside the Jupyter Application, locate the New button on the top right corner, then select Python 3 from the drop down menu.

Cell Types

The text that you type in this .ipynb file will be organized into cells, and there are different types of cells that you can use depending on what you want to do. To create a new cell, click the + button in the top left corner. The buttons to the right of it are used for deleting cells, copying and pasting cells, and for moving them up and down.

Code

Code cells are used for python code. These cells will look a lot like iPython, with the same In[1]: headers. Simply type your input code in the cell, then click the **Run** button to generate a corresponding Out[]: cell that displays the result, if it exists (e.g. typing an import statement should not return anything). In the context of pandas, if we type code that generates a Dataframe into a code cell, when we run it, Jupyter generates a really nice table representation of what the Dataframe looks like.

In [6]:
```
summary = df.describe()
summary
```

Out[6]:

	Total	Hp	Attack	Defense	Sp.Atk	Sp.Def	Speed	Comb. Attack	Comb. Defense
count	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000	95.000000
mean	419.936842	69.273684	76.589474	71.326316	67.831579	70.189474	64.726316	144.421053	141.515789
std	99.799993	22.789248	30.038803	29.790203	28.002527	25.816989	26.742047	46.776194	50.073109
min	194.000000	20.000000	5.000000	5.000000	10.000000	30.000000	5.000000	20.000000	60.000000
25%	330.000000	55.000000	56.500000	48.500000	46.000000	50.000000	40.000000	110.000000	101.000000
50%	440.000000	68.000000	76.000000	67.000000	62.000000	65.000000	61.000000	145.000000	140.000000
75%	510.000000	79.500000	94.500000	87.000000	86.500000	85.500000	85.000000	184.000000	177.500000
max	600.000000	150.000000	165.000000	168.000000	135.000000	150.000000	125.000000	230.000000	306.000000

Markdown

Markdown cells are used for text. In most cases, we use markdown cells to add text to give context to our code cells. These cells are also really nice in that they include a few formatting features like headers, display code, and many others. After typing out your text, click **Run** or press **shift + enter** to execute the cell. Click this link to view a reference guide for MarkDown styling.

Headers, Italics, and Bolding

To signify a line as a header line, use the # character. One # denotes the largest size header, and each additional # denotes a slightly smaller header (### is a smaller header than #).

```
# This is a big header

### This is a smaller header

This is just ordinary text<br/>
*This is italicized text*<br/>
**This is bolded text**<br/>
```

In edit Mode

This is a big header

This is a smaller header

This is just ordinary text
This is italicized text
This is bolded text

*After **Run***

Display Code

If you have code that is only meant for display purposes (there is no need to actually execute it) and you want to include it in a markdown cell, use the following notation with backticks (use the keyboard key with the ~ on it next to 1), and specify the language that the code will be in. The backtick is NOT the same as the single quote character)

```
```python
def func():
    print("Hello!")
```
```

In edit Mode

```python
def func():
    print("Hello!")
```

*After **Run***